```
buff = save(syn_model)
reloaded = load(buff)

assert syn_model.name() == reloaded.name()
```

- Saving and loading models from disk

```
from sklearn.datasets import load_diabetes
from synthcity.utils.serialization import save_to_file, load_from_file
from synthcity.plugins import Plugins

X, y = load_diabetes(return_X_y=True, as_frame=True)
X["target"] = y

syn_model = Plugins().get("adsgan", n_iter=10)

syn_model.fit(X)

save_to_file('./adsgan_10_epochs.pkl', syn_model)
reloaded = load_from_file('./adsgan_10_epochs.pkl')

assert syn_model.name() == reloaded.name()
```

- Using the Serializable interface

```
from synthcity.plugins import Plugins

syn_model = Plugins().get("adsgan")

buff = syn_model.save()
reloaded = Plugins().load(buff)

assert syn_model.name() == reloaded.name()
```

📖 Tutorials

- Open in Colab Tutorial 0: Getting started with tabular data
- Open in Colab Tutorial 1: Writing a new plugin
- Open in Colab Tutorial 2: Benchmarking models
- Open in Colab Tutorial 3: Generating Survival Analysis data
- Open in Colab Tutorial 4: Generating Time Series
- Open in Colab Tutorial 5: Generating Data with Differential Privacy Guarantees
- Open in Colab Tutorial 6: Practice using Custom Time series data
- Open in Colab Tutorial 7: Image generation
- Open in Colab Tutorial 8: hyperparameter optimization
- Open in Colab Tutorial 9: dealing with missing data

🔑 Methods

Bayesian methods

Method	Description	Reference
bayesian_network	The method represents a set of random variables and their conditional dependencies via a directed acyclic graph (DAG), and uses it to sample new data points	pgmpy
privbayes	A differentially private method for releasing high-dimensional data.	PrivBayes: Private Data Release via Bayesian Networks

Generative adversarial networks(GANs)

Method	Description	Reference
adsgan	A conditional GAN framework that generates synthetic data while minimize patient identifiability that is defined based on the probability of re-identification given the combination of all data on any individual patient	Anonymization Through Data Synthesis Using Generative Adversarial Networks (ADS-GAN)
pategan	The methos uses the Private Aggregation of Teacher Ensembles (PATE) framework and applies it to GANs, allowing to tightly bound the influence of any individual sample on the model, resulting in tight differential privacy guarantees and thus an improved performance over models with the same guarantees.	PATE-GAN: Generating Synthetic Data with Differential Privacy Guarantees
ctgan	A conditional generative adversarial network which can handle tabular data.	Modeling Tabular data using Conditional GAN

Variational autoencoders(VAE)

Method	Description	Reference
tvae	A conditional VAE network which can handle tabular data.	Modeling Tabular data using Conditional GAN
rtvae	A robust variational autoencoder with β divergence for tabular data (RTVAE) with mixed categorical and continuous features.	Robust Variational Autoencoder for Tabular Data with β Divergence

Normalizing Flows

Method	Description	Reference
nflow	Normalizing Flows are generative models which produce tractable distributions where both sampling and density evaluation can be efficient and exact.	Neural Spline Flows

Graph neural networks

README Code of conduct Contributing Apache-2.0 license

goggle	GOGGLE: Generative Modelling for Tabular Data by Learning Relational Structure	Paper

Diffusion models

Method	Description	Reference
ddpm	TabDDPM: Modelling Tabular Data with Diffusion Models.	Paper

Random Forest models

Method	Description	Reference
arfpy	Adversarial Random Forests for Density Estimation and Generative Modeling	Paper

LLM-based models

Method	Description	Reference
GReaT	Language Models are Realistic Tabular Data Generators	Paper

Static Survival analysis methods

Method	Description	Reference
survival_gan	SurvivalGAN is a generative model that can handle survival data by addressing the imbalance in the censoring and time horizons, using a dedicated mechanism for approximating time to event/censoring from the input and survival function.	---
survival_ctgan	SurvivalGAN version using CTGAN	---
survae	SurvivalGAN version using VAE	---
survival_nflow	SurvivalGAN version using normalizing flows	---

Time-Series and Time-Series Survival Analysis methods

Method	Description	Reference
timegan	TimeGAN is a framework for generating realistic time-series data that combines the flexibility of the unsupervised paradigm with the control afforded by supervised training. Through a learned embedding space jointly optimized with both supervised and adversarial objectives, the network adheres to the dynamics of the training data during sampling.	Time-series Generative Adversarial Networks
fflows	FFlows is an explicit likelihood model based on a novel class of normalizing flows that view time-series data in the frequency-domain rather than the time-domain. The method uses a discrete Fourier transform (DFT) to convert variable-length time-series with arbitrary sampling periods into fixed-length spectral representations, then applies a (data-dependent) spectral filter to the frequency-transformed time-series.	Generative Time-series Modeling with Fourier Flows

Privacy & Fairness

Method	Description	Reference
decaf	Machine learning models have been criticized for reflecting unfair biases in the training data. Instead of solving this by introducing fair learning algorithms directly, DEACF focuses on generating fair synthetic data, such that any downstream learner is fair. Generating fair synthetic data from unfair data - while remaining truthful to the underlying data-generating process (DGP) - is non-trivial. DECAF is a GAN-based fair synthetic data generator for tabular data. With DECAF, we embed the DGP explicitly as a structural causal model in the input layers of the generator, allowing each variable to be reconstructed conditioned on its causal parents. This procedure enables inference time debiasing, where biased edges can be strategically removed to satisfy user-defined fairness requirements.	DECAF: Generating Fair Synthetic Data Using Causally-Aware Generative Networks
privbayes	A differentially private method for releasing high-dimensional data.	PrivBayes: Private Data Release via Bayesian Networks
dpgan	Differentially Private GAN	Differentially Private Generative Adversarial Network
adsgan	A conditional GAN framework that generates synthetic data while minimize patient identifiability that is defined based on the probability of re-identification given the combination of all data on any individual patient	Anonymization Through Data Synthesis Using Generative Adversarial Networks (ADS-GAN)
pategan	The methos uses the Private Aggregation of Teacher Ensembles (PATE) framework and applies it to GANs, allowing to tightly bound the influence of any individual sample on the model, resulting in tight differential privacy guarantees and thus an improved performance over models with the same guarantees.	PATE-GAN: Generating Synthetic Data with Differential Privacy Guarantees

Domain adaptation

Method	Description	Reference
radialgan	Training complex machine learning models for prediction often requires a large amount of data that is not always readily available. Leveraging these external datasets from related but different sources is, therefore, an essential task if good predictive models are to be built for deployment in settings where data can be rare. RadialGAN is an approach to the problem in which multiple GAN architectures are used to learn to translate from one dataset to another, thereby allowing to augment the target dataset effectively and learning better predictive models than just the target dataset.	RadialGAN: Leveraging multiple datasets to improve target-specific predictive models using Generative Adversarial Networks

Images

Method	Description	Reference
image_cgan	Conditional GAN for generating images	---
image_adsgan	The AdsGAN method adapted for image generation	---

Debug methods

Method	Description	Reference
marginal_distributions	A differentially private method that samples from the marginal distributions of the training set	---
uniform_sampler	A differentially private method that uniformly samples from the [min, max] ranges of each column.	---
dummy_sampler	Resample data points from the training set	---

Evaluation metrics

The following table contains the available evaluation metrics:

- **Sanity checks**

Metric	Description	Values
data_mismatch	Average number of columns with datatype(object, real, int) mismatch between the real and synthetic data	0: no datatype mismatch. 1: complete data type mismatch between the datasets.
common_rows_proportion	The proportion of rows in the real dataset leaked in the synthetic dataset.	0: there are no common rows between the real and synthetic datasets. 1: all the rows in the real dataset are leaked in the synthetic dataset.
nearest_syn_neighbor_distance	Average distance from the real data to the closest neighbor in the synthetic data	0: all the real rows are leaked in the synthetic dataset. 1: all the synthetic rows are far away from the real dataset.
close_values_probability	The probability of close values between the real and synthetic data.	0: there is no chance to have synthetic rows similar to the real. 1 means that all the synthetic rows are similar to some real rows.
distant_values_probability	Average distance from the real data to the closest neighbor in the synthetic data	0: no chance to have rows in the synthetic far away from the real data. 1: all the synthetic datapoints are far away from the real data.

- **Statistical tests**

Metric	Description	Values
inverse_kl_divergence	The average inverse of the Kullback–Leibler Divergence	0: the datasets are from different distributions. 1: the datasets are from the same distribution.
ks_test	The Kolmogorov-Smirnov test	0: the distributions are totally different. 1: the distributions are identical.
chi_squared_test	The p-value. A small value indicates that we can reject the null hypothesis and that the distributions are different.	0: the distributions are different 1: the distributions are identical.
max_mean_discrepancy	Empirical maximum mean discrepancy.	0: The distributions are the same. 1: The distributions are totally different.
jensenshannon_dist	The Jensen-Shannon distance (metric) between two probability arrays. This is the square root of the Jensen-Shannon divergence.	0: The distributions are the same. 1: The distributions are totally different.

Metric	Description	Values
wasserstein_dist	Wasserstein Distance is a measure of the distance between two probability distributions.	0: The distributions are the same.
prdc	Computes precision, recall, density, and coverage given two manifolds.	---
alpha_precision	Evaluate the alpha-precision, beta-recall, and authenticity scores.	---
survival_km_distance	The distance between two Kaplan-Meier plots(survival analysis).	---
fid	The Frechet Inception Distance (FID) calculates the distance between two distributions of images.	---

- **Synthetic Data quality**

Metric	Description	Values
performance.xgb	Train an XGBoost classifier/regressor/survival model on real data(gt) and synthetic data(syn), and evaluate the performance on the test set.	1 for ideal performance, 0 for worst performance
performance.linear	Train a Linear classifier/regressor/survival model on real data(gt) and the synthetic data and evaluate the performance on test data.	1 for ideal performance, 0 for worst performance
performance.mlp	Train a Neural Net classifier/regressor/survival model on the real data and the synthetic data and evaluate the performance on test data.	1 for ideal performance, 0 for worst performance
performance.feat_rank_distance	Train a model on the synthetic data and a model on the real data. Compute the feature importance of the models on the same test data, and compute the rank distance between the importance(kendalltau or spearman)	1: similar ranks in the feature importance. 0: uncorrelated feature importance
detection_gmm	Train a GaussianMixture model to differentiate the synthetic data from the real data.	0: The datasets are indistinguishable. 1: The datasets are totally distinguishable.
detection_xgb	Train an XGBoost model to differentiate the synthetic data from the real data.	0: The datasets are indistinguishable. 1: The datasets are totally distinguishable.
detection_mlp	Train a Neural net to differentiate the synthetic data from the real data.	0: The datasets are indistinguishable. 1: The datasets are totally distinguishable.
detection_linear	Train a Linear model to differentiate the synthetic data from the real data.	0: The datasets are indistinguishable. 1: The datasets are totally distinguishable.

- **Privacy metrics**

Quasi-identifiers : pieces of information that are not of themselves unique identifiers, but are sufficiently well correlated with an entity that they can be combined with other quasi-identifiers to create a unique identifier.

Metric	Description	Values
k_anonymization	The minimum value k which satisfies the k-anonymity rule: each record is similar to at least another k-1 other records on the potentially identifying variables.	Reported on both the real and synthetic data.
l_diversity	The minimum value l which satisfies the l-diversity rule: every generalized block has to contain at least l different sensitive values.	Reported on both the real and synthetic data.
kmap	The minimum value k which satisfies the k-map rule: every combination of values for the quasi-identifiers appears at least k times in the reidentification(synthetic) dataset.	Reported on both the real and synthetic data.

Metric	Description	Values
delta_presence	The maximum re-identification risk for the real dataset from the synthetic dataset.	0 for no risk.
identifiability_score	The re-identification score on the real dataset from the synthetic dataset.	---]
sensitive_data_reidentification_xgb	Sensitive data prediction from the quasi-identifiers using an XGBoost.	0 for no risk.
sensitive_data_reidentification_mlp	Sensitive data prediction from the quasi-identifiers using a Neural Net.	0 for no risk.

🔍 Use-cases of synthetic data beyond fidelity & privacy

The following table contains research projects and papers that have been completed using Synthcity. Feel free to explore them and be inspired to use Synthcity in your own research project!

Project	Description	Code	Paper
Synthetic data capturing nuances of real data	Ensuring that synthetic data mirrors the complex nuances of real-world data is a challenging task. This paper addresses this issue by exploring the potential of integrating data-centric AI techniques which profile the data to guide the synthetic data generation process.	Code	NeurIPS 2023 paper
Model Evaluation/Testing	This paper aim to improve model evaluation with synthetic test data	Code	NeurIPS 2023 paper
Generative uncertainty	This paper explores how the generative process affects the downstream ML task.	Code	ICML 2023 paper
Benchmarking Synthcity	This paper employs synthcity's benchmarking pipeline to compare different SOTA methods across multiple openml datasets. The paper aims to introduce the wider ML community to Synthcity.	Code	NeurIPS 2023 paper

🔨 Tests

Install the testing dependencies using

```
pip install .[testing]
```

The tests can be executed using

```
pytest -vsx
```

Contributing to Synthcity

We want to make contributing to Synthcity is as easy and transparent as possible. We hope to collaborate with as many people as we can.

Development installation

First create a new environment. It is recommended that you use conda. This can be done as follows:

```
conda create -n your-synthcity-env python=3.9
conda activate your-synthcity-env
```

Python versions 3.7, 3.8, 3.9, and 3.10 are all compatible, but it is best to use the most up to date version you can, as some models may not support older python versions.

To get the development installation with all the necessary dependencies for linting, testing, auto-formatting, and pre-commit etc. run the following:

```
git clone https://github.com/vanderschaarlab/synthcity.git
cd synthcity
pip install -e .[testing]
```